Low Level Design On

Containerized Marketing Service with Docker

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Bhavya Rathore	EN22CS301269
2	Bhoomika Sanodiya	EN22CS301271
3	Bhumika Chouhan	EN22CS301275
4	Devi Sanjaykumar Modi	EN23CS3T1008
5	Dollyna Sahu	EN22CS301354

Group Name: Group G2D3

Project Number: DO-26

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Prof. Shyam Patel

Academic Year: 2025-2026

Table of Contents

1. Introduction.
    - 1.1. Scope of the document.
    - 1.2. Intended Audience
    - 1.3. System overview.
2. System Design.
    - 2.1. High Level Architecture
    - 2.2. Module Design
    - 2.3. Data flow Architecture
    - 2.4. System Workflow
    - 2.5. API Catalogue
3. References

1. Introduction

The rapid growth of web-based applications has increased the demand for scalable, maintainable, and easily deployable backend systems. Modern software development practices emphasize modular architecture, containerization, and automation to ensure consistent performance across different environments such as development, testing, and production.

This project focuses on designing and implementing a Marketing Service using PHP Laravel, which is a powerful and widely used backend framework. The system is designed to handle marketing-related operations such as campaign management, user interaction tracking, content handling, and API-based communication with external services.

To ensure portability, consistency, and ease of deployment, the entire application is containerized using Docker. Docker enables the application and its dependencies to run in isolated environments, eliminating issues caused by environment differences between developers' machines and production servers. Additionally, Docker Compose is used to orchestrate multiple services such as the application server, web server, and database in a unified configuration.

1.1 Scope of the Document

This document provides the Low Level Design (LLD) of a containerized Marketing Service developed using the PHP Laravel framework. The system is designed with a strong focus on modular architecture, scalability, maintainability, and portability using Docker-based containerization.

The scope of this document includes:

- Detailed internal structure of the Laravel application
- Module-wise breakdown of system components
- Interaction between application layers
- Data flow across the system
- Docker-based containerization strategy
- Docker Compose orchestration of multiple services
- Deployment considerations for development and production environments

The objective is to ensure that the system is environment-independent and can run consistently across all stages of the software development lifecycle.

1.2 Intended Audience

This document is intended for:

- Backend developers working on Laravel applications
- DevOps engineers managing containerized deployments
- System architects responsible for scalable system design
- QA engineers validating environment consistency and functionality
- Project evaluators and interviewers assessing system design and DevOps practices
- Future maintainers and contributors of the project

1.3 System Overview

The Marketing Service is a backend system built using PHP Laravel that supports marketing automation and management functionalities such as campaign creation, content handling, user interaction tracking, and API-based integration with external systems.

The system is designed to ensure:

- High portability across environments
- Easy scalability for future enhancements
- Modular separation of concerns
- Secure and consistent execution environment
- Simplified deployment using containerization

To achieve these goals, the system is fully containerized using Docker and orchestrated using Docker Compose. This ensures that all services (application, database, and web server) work together seamlessly while remaining isolated from each other.

The architecture follows industry best practices such as separation of application layers, use of service-based design, and optimized container builds to improve performance and maintainability.

2. System Design

2.1 High-Level Architecture

The system follows a layered containerized architecture that separates concerns into distinct logical units:

1. Presentation Layer (Web Interface/API Layer)

Handles incoming HTTP requests from users or external systems. It acts as the entry point of the application and communicates with backend services.

2. Application Layer (Laravel Backend)

This is the core of the system where business logic is implemented. It is responsible for processing requests, executing business rules, and generating responses.

3. Service Layer (Business Logic Layer)

This layer isolates core business logic from controllers to ensure reusability, maintainability, and easier testing.

4. Data Layer (Database Layer)

Responsible for persistent storage of all application data including campaigns, users, logs, and analytics information.

5. Containerization Layer (Docker)

Provides isolated runtime environments for all components of the system, ensuring consistency across development, staging, and production environments.

6. Orchestration Layer (Docker Compose)

Manages and coordinates multiple containers, ensuring proper startup order, service communication, and persistent storage handling.

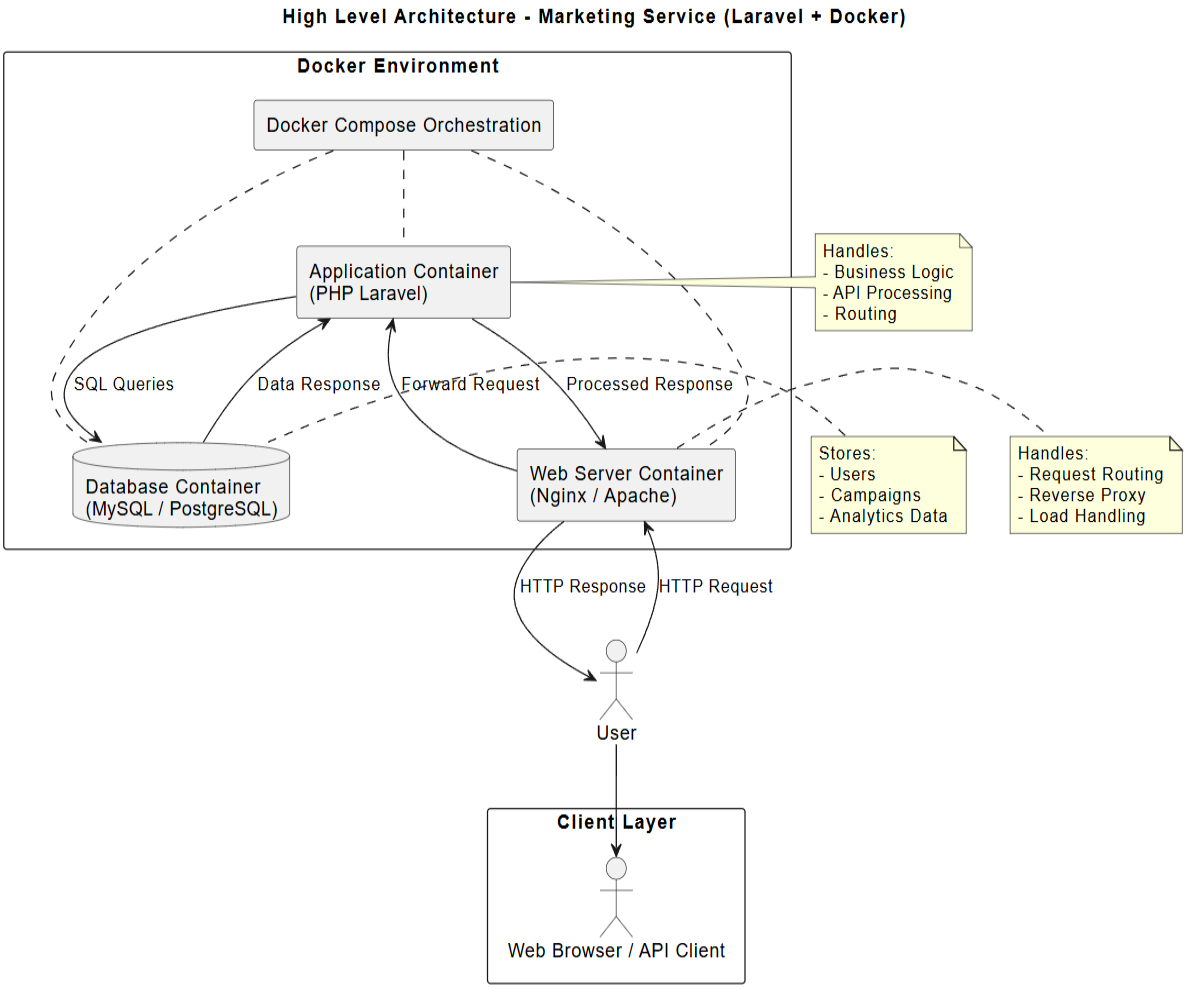

figure 1: High-level Architecture

2.2 Module Design (Laravel Application)

1. Controller Layer

Acts as the entry point for all API requests. It is responsible for:

- Receiving HTTP requests
- Validating input data
- Calling appropriate service methods
- Returning responses

Controllers are kept lightweight to maintain clean architecture.

2. Service Layer

The service layer contains the core business logic of the application. It:

- Processes marketing-related operations
- Applies business rules and validations
- Acts as an intermediary between controllers and models
- Ensures reusability of logic across multiple modules.

3. Model Layer

Represents database structure and handles data operations. It:

- Maps database tables to objects
- Performs CRUD operations
- Maintains relationships between entities

4. Repository Layer (Optional but Recommended)

This layer abstracts database operations from business logic. It:

- Improves code maintainability
- Supports unit testing
- Enables easy database switching in future

5. Routing Module

Defines application endpoints and maps them to controllers. It:

- Organizes API structure
- Supports versioning of APIs
- Ensures clean request handling flow

2.3 Data Flow Architecture

The system follows a structured request-response flow:

1. Client sends an HTTP request
2. Web server receives the request and forwards it to Laravel
3. Laravel routing system identifies the correct controller

4. Controller validates input and calls service layer
5. Service layer executes business logic
6. Model layer interacts with the database
7. Response is generated and sent back through the same path

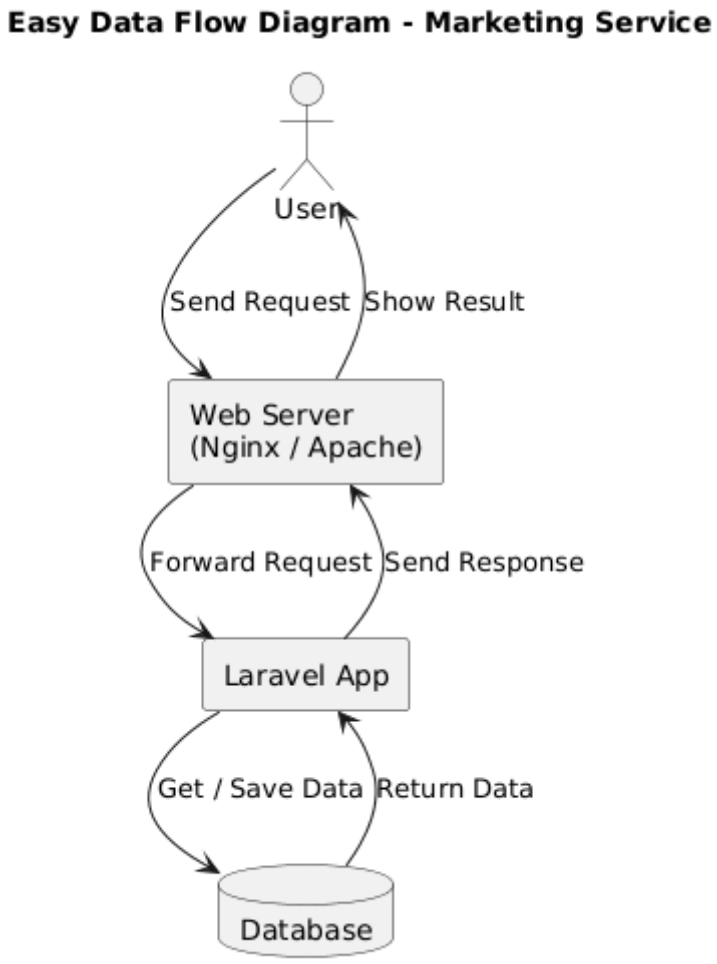

figure 2: Data Flow Diagram

2.4 System Workflow

- User or external system sends a request to the API
- Web server processes and forwards the request
- Laravel application handles request routing

- Business logic is executed in the service layer
- Database operations are performed if required
- Structured response is returned to the client
- Logs are generated

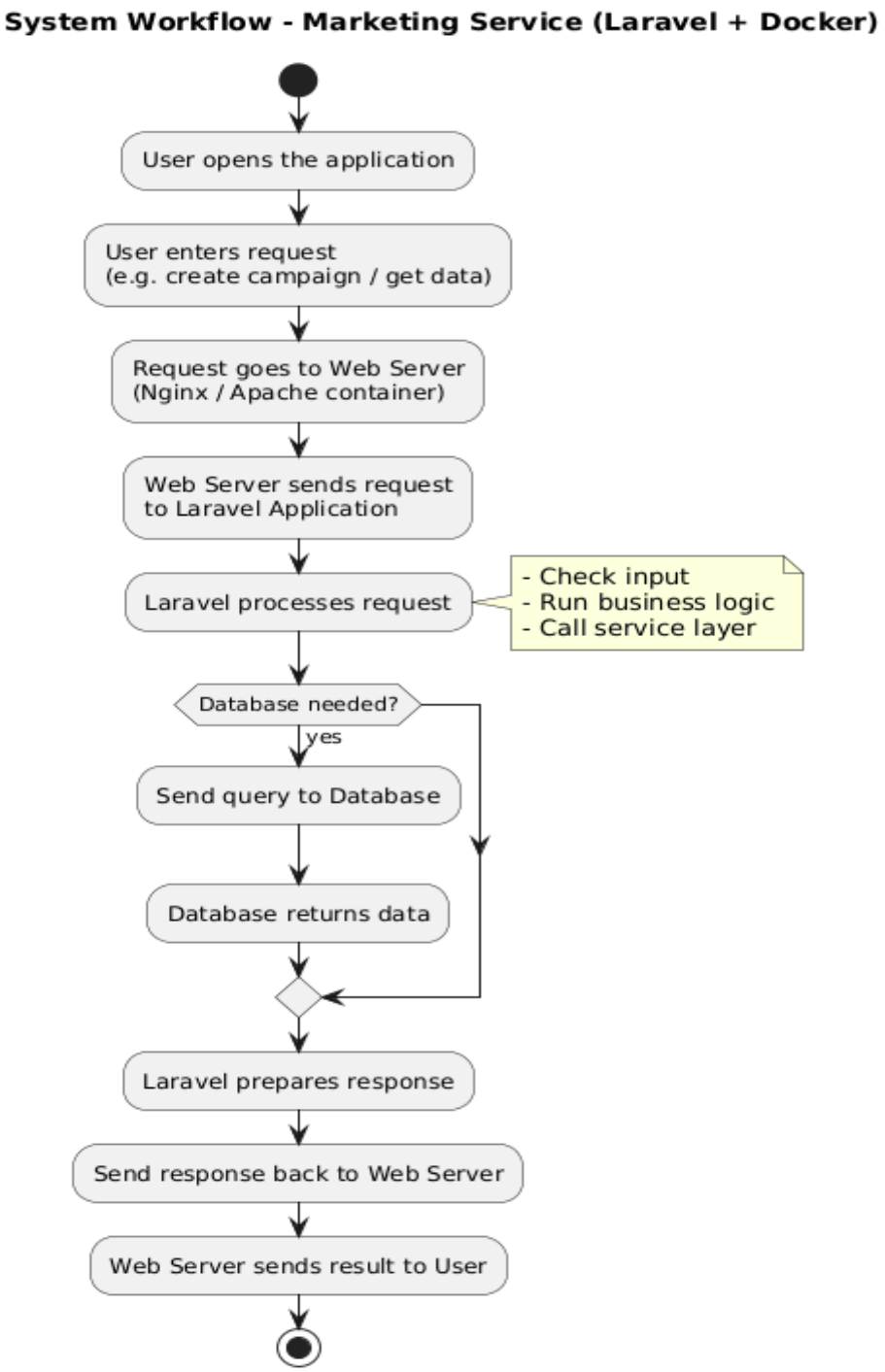

figure 3: System workflow diagram

2.5 API Catalogue – Marketing Service (Laravel)

The API Catalogue defines all the available REST APIs in the Marketing Service system. These APIs are used to perform operations such as creating marketing campaigns, retrieving campaign data, updating information, and managing user-related marketing activities.

All APIs follow REST principles and communicate using JSON format. The system is containerized using Docker, ensuring consistent API behavior across development and production environments.

Base URL

All APIs are accessed through a base URL depending on the environment:

- Development: `http://localhost:8080/api`
- Production: `https://<domain-name>/api`

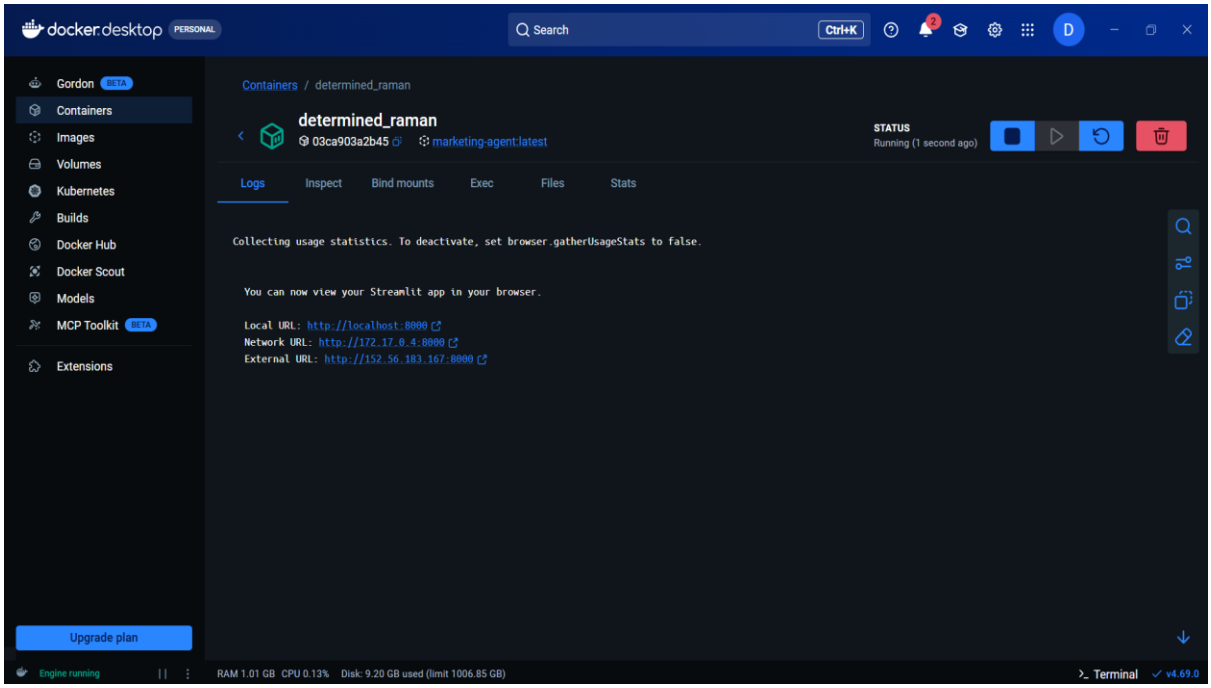

figure 4: Screenshot of Docker Integration

3. References

[1] Laravel Documentation, “Laravel Framework Documentation,” 2026. [Online]. Available: https://laravel.com/docs.

[2] Docker Documentation, “Docker Overview and Guides,” 2026. [Online]. Available: https://docs.docker.com.

[3] Docker Inc., “Docker Compose Documentation,” 2026. [Online]. Available: https://docs.docker.com/compose/.

[4] MDN Web Docs, “HTTP Status Codes,” 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status.

[5] MySQL Documentation, “MySQL 8.0 Reference Manual,” 2026. [Online]. Available: https://dev.mysql.com/doc/.

[6] Microsoft Learn, “REST API Design Guidelines,” 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design.